

Engineering the Intervention

Session 3

2026-09-15

Today

- › **The bundle, in code**: which of the seven layers become software
- › **The pieces**: a gateway, an agent framework, a web API and page, an eval framework
 - » Which pieces a human-subjects team builds, and which a model-only team builds
- › **Deployment**: where the data goes, and what constrains where it *may* go
- › **Your system**: what to prepare for next week and over the next *few* weeks



The bundle, in code



The tutor bundle from last week

Layer	Setting	Today
Weights	A model id at a gateway, pinned	chosen in code
Decoding	Temperature, output budget, reasoning on or off	written in code
Prompt	Instructions that withhold the answer; per-arm text; history	written in code
Harness	Memory window, retries, what counts as the reply	written in code
Interface	A chat page inside the survey, logging arm and participant	written in code
Workflow	Link unlocks 48 hours before the deadline	a design decision
Organization	Course policy permits the helper	given



The tutor bundle from last week

Layer	Setting	Today
Weights	A model id at a gateway, pinned	chosen in code
Decoding	Temperature, output budget, reasoning on or off	written in code
Prompt	Instructions that withhold the answer; per-arm text; history	written in code
Harness	Memory window, retries, what counts as the reply	written in code
Interface	A chat page inside the survey, logging arm and participant	written in code
Workflow	Link unlocks 48 hours before the deadline	a design decision
Organization	Course policy permits the helper	given

Much of last week's table becomes a configuration object in our codebase.



Which pieces each archetype builds

Piece	Human-subjects	Model-only
Gateway and model	yes	yes
Agent: instructions, settings, the arm	yes	yes
Harness: memory, retries, logging	yes	yes
Web API and page, deployed	yes	usually no
Task set with ground truth, scorers	maybe	the study itself



Which pieces each archetype builds

Piece	Human-subjects	Model-only
Gateway and model	yes	yes
Agent: instructions, settings, the arm	yes	yes
Harness: memory, retries, logging	yes	yes
Web API and page, deployed	yes	usually no
Task set with ground truth, scorers	maybe	the study itself

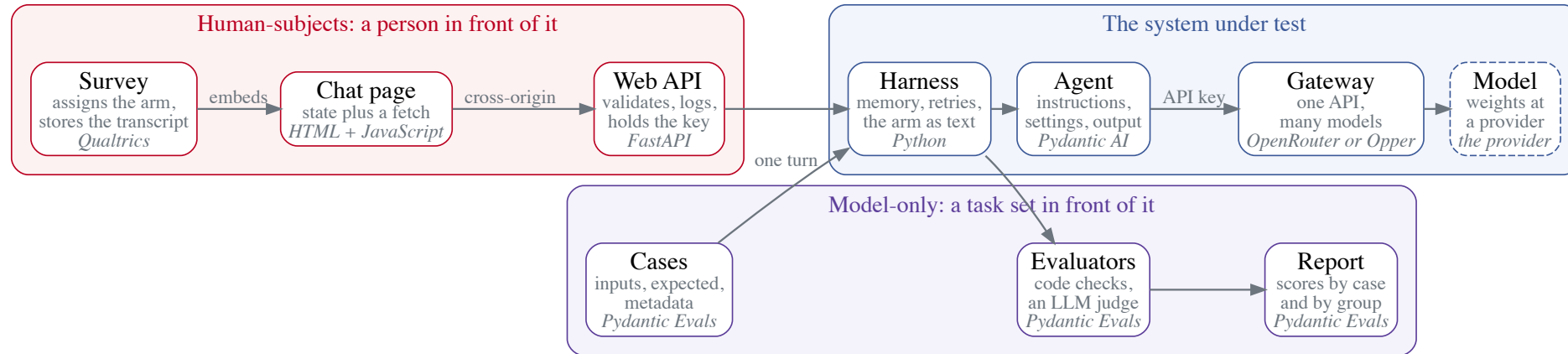
- › Both archetypes build the same underlying system.
- › A human-subjects team may run the offline check before fielding



The pieces and how they fit



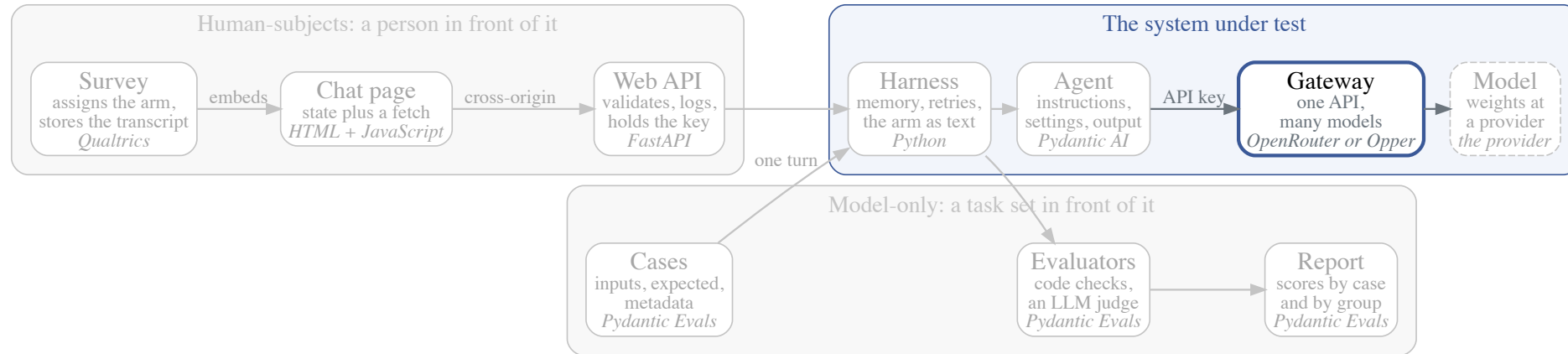
One picture



- › The **system under test** is the same code in both archetypes.



The gateway



A gateway: one API in front of many models

- › One request shape, hundreds of models, a **model id string** to select a model
- › The weights layer becomes a config value
- › The gateway routes the request to a provider under your constraints
 - » Last week's warnings on reproducibility are important to remember



A gateway: one API in front of many models

- › One request shape, hundreds of models, a **model id string** to select a model
- › The weights layer becomes a config value
- › The gateway routes the request to a provider under your constraints
 - » Last week's warnings on reproducibility are important to remember

Two gateways depending on where you are in development:

- › **Iteration:** OpenRouter's free ids, a flag to refuse providers who train on prompts.
 - » Writing prompts, wiring the API, running the pipeline. No sensitive data whatsoever.
- › **Production:** Opper, EU-hosted, zero data retention, a monthly cap per team.
 - » The study, and any run you will report



Environment variables: secrets and configuration

An **environment variable** is a named value available to a running program. A `.env` file supplies those values during local development.

```
# lecture-slides/.env – local file, excluded from Git
OPENROUTER_API_KEY=your-key-here
OPPER_API_KEY=your-other-key-here
APP_TOKEN=your-app-token-here          # optional: browser sends X-App-Token
TUTOR_BACKEND=openrouter
TUTOR_MODEL=cohere/north-mini-code:free
ALLOWED_ORIGINS=https://your-survey.qualtrics.com
```

```
import os
from tutor import load_env
load_env() # demo helper: loads .env into the process environment
OPENROUTER_API_KEY = os.environ["OPENROUTER_API_KEY"]
```



Environment variables: secrets and configuration

An **environment variable** is a named value available to a running program. A `.env` file supplies those values during local development.

```
# lecture-slides/.env – local file, excluded from Git
OPENROUTER_API_KEY=your-key-here
OPPER_API_KEY=your-other-key-here
APP_TOKEN=your-app-token-here          # optional: browser sends X-App-Token
TUTOR_BACKEND=openrouter
TUTOR_MODEL=cohere/north-mini-code:free
ALLOWED_ORIGINS=https://your-survey.qualtrics.com
```

```
import os
from tutor import load_env
load_env() # demo helper: loads .env into the process environment
OPENROUTER_API_KEY = os.environ["OPENROUTER_API_KEY"]
```

- › `.env` is plain text: keep it out of Git, slides, browser code, and shared outputs
- › Gateway keys stay on the server; `APP_TOKEN` is also supplied to the browser
- › When deploying, enter these values in the host's environment-variable settings



Choosing the weights

The **weights** layer: choose a model id and the gateway that serves it.

```
from pydantic_ai.models.openrouter import OpenRouterModel
from pydantic_ai.providers.openrouter import OpenRouterProvider

model = OpenRouterModel(
    "cohere/north-mini-code:free", # which weights
    provider=OpenRouterProvider(api_key=OPENROUTER_API_KEY),
)
```



Choosing the weights

The **weights** layer: choose a model id and the gateway that serves it.

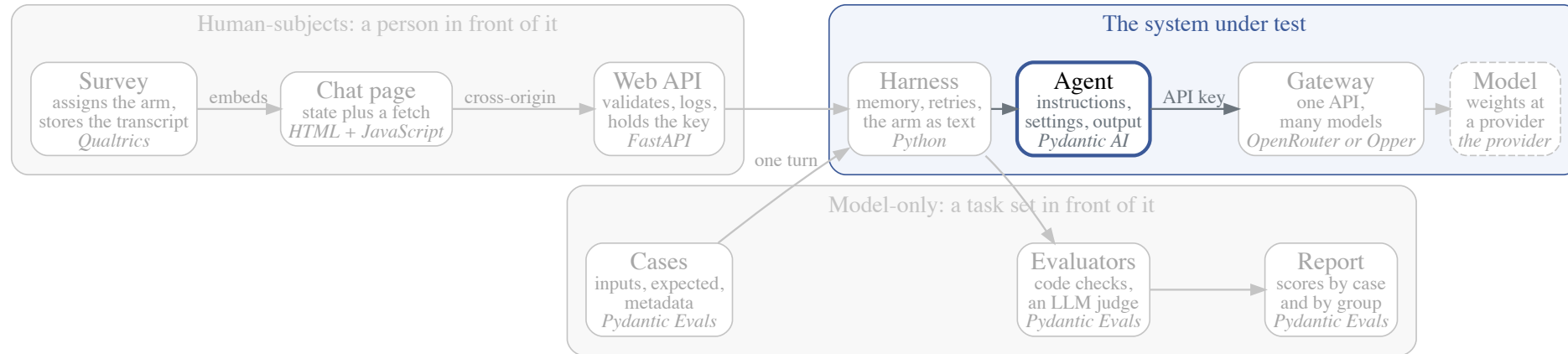
```
from pydantic_ai.models.openrouter import OpenRouterModel
from pydantic_ai.providers.openrouter import OpenRouterProvider

model = OpenRouterModel(
    "cohere/north-mini-code:free", # which weights
    provider=OpenRouterProvider(api_key=OPENROUTER_API_KEY),
)
```

- › The demo wraps this in `make_model()` : the configured backend supplies the model



The agent



The tutor agent: a model plus instructions

The tutor agent brings together three layers from last week's bundle.

- › **Weights** → the model we just selected
- › **Decoding** → model settings: temperature, token budget, reasoning on or off
- › **Prompt** → system prompt for the tutor's role; instructions for how to teach
- › Send a student's question; get back the tutor's reply, usage, and message history



The tutor agent: a model plus instructions

The tutor agent brings together three layers from last week's bundle.

- › **Weights** → the model we just selected
- › **Decoding** → model settings: temperature, token budget, reasoning on or off
- › **Prompt** → system prompt for the tutor's role; instructions for how to teach
- › Send a student's question; get back the tutor's reply, usage, and message history
- › Dependencies carry per-run context into the instructions
- › Here, a **MyDeps** dataclass carries one field: how the tutor should help the student
- › System prompts persist in message history; instructions are recomputed for each run



Async / await: let other requests run while we wait

Like a waiter taking another table's order while the kitchen prepares yours.

```
async def ask(agent, question):           # this function can pause
    result = await agent.run(question)    # wait here for the model's reply
    return result.output                  # continue when the reply arrives
```

- › **async def** : declares a function that can give control back while waiting
 - » **await** : this function waits for the result; other scheduled work can run meanwhile
 - » You can't use the output of **ask** until you have **await** ed it
- › For our API: another student's request can progress while the model answers this one



Async / await: let other requests run while we wait

Like a waiter taking another table's order while the kitchen prepares yours.

```
async def ask(agent, question):           # this function can pause
    result = await agent.run(question)    # wait here for the model's reply
    return result.output                  # continue when the reply arrives
```

- › **async def** : declares a function that can give control back while waiting
 - » **await** : this function waits for the result; other scheduled work can run meanwhile
 - » You can't use the output of **ask** until you have **await** ed it
- › For our API: another student's request can progress while the model answers this one
- › The lines within **ask** still run in order. **await** does not make the model answer faster
- › In the notebook, use **await ask(agent, "What is a logit?")** ; FastAPI runs our async endpoint for us



The tutor agent, in code

```
from dataclasses import dataclass
from pydantic_ai import Agent, RunContext
from tutor import make_settings

@dataclass
class MyDeps:
    tutoring_style: str

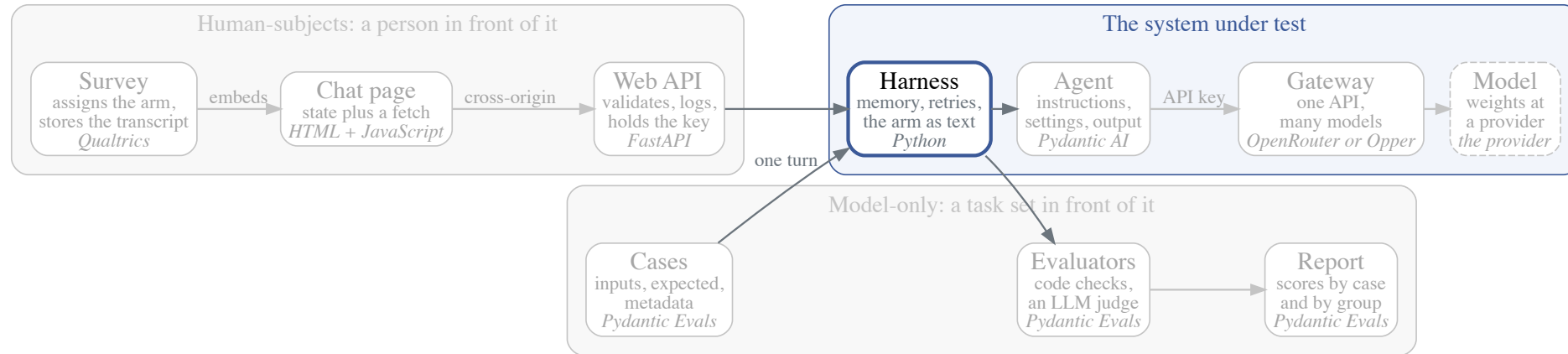
settings = make_settings(temperature=0.7, max_tokens=800, reasoning=False)
agent = Agent(model, model_settings=settings, deps_type=MyDeps,
    system_prompt="You are a statistics tutor.", # prompt: role
    instructions="Help the student work it out; keep replies brief.")

@agent.instructions # reads a field from this run's dependencies
def style(ctx: RunContext[MyDeps]) -> str:
    return ctx.deps.tutoring_style

result = await agent.run("How do I start Q3?",
    deps=MyDeps(tutoring_style="Guide with questions; do not give the answer.
```



The harness



The harness: controlling how the tutor runs

We configured the agent's model, decoding, and prompt. The **harness** is the execution logic around model calls:

- › **Memory**: select which earlier turns or retrieved material enter the next call
- › **Tools**: expose allowed functions, execute the model's requested calls, return their results
- › **Control flow**: retry failures, make another model call, or stop at a limit
- › **Logging**: record the messages, actions, model, tokens, and cost



The harness: controlling how the tutor runs

We configured the agent's model, decoding, and prompt. The **harness** is the execution logic around model calls:

- › **Memory**: select which earlier turns or retrieved material enter the next call
- › **Tools**: expose allowed functions, execute the model's requested calls, return their results
- › **Control flow**: retry failures, make another model call, or stop at a limit
- › **Logging**: record the messages, actions, model, tokens, and cost
- › Pydantic AI's **Agent** implements some harness behavior, including the tool-call loop; our **Tutor** adds history selection, retries, and records
- › This tutor has no tools. Adding a calculator or course-note search would change its harness



The harness, in code

```
@dataclass
class TutorConfig:                                # the bundle, as fields
    model: str | None = None                        # weights
    temperature: float = 0.7                       # decoding
    reasoning: bool = False                        # decoding
    system_prompt: str = SYSTEM_PROMPT              # prompt: role
    instructions: str = INSTRUCTIONS                # prompt: teaching rules
    condition_instructions: dict[str, str] = ...    # prompt, per arm
    max_history_turns: int = 10                     # harness

class Tutor:
    async def reply(self, history, user_message, session) -> Reply:
        message_history = self.build_history(history) # memory window
        result = await run_with_retries(lambda: self.agent.run(
            user_message, deps=session, message_history=message_history))
        return Reply(text=result.output, model=result.response.model_name,
            cost_usd=result.usage.cost,
            messages=result.all_messages())
```



Where the bundle's layers live

Layer	Object	What varying it means
Weights	a model id, resolved at a backend	another model, another vendor, another lane
Decoding	model settings	temperature, budget, reasoning
Prompt	agent instructions, plus per-arm text keyed by the arm	the treatment, in most human-subjects designs
Harness	agent run loop + the Tutor wrapper	memory, tools, retries, stopping rules, logs
Interface	the web API and the page	the layer a participant actually touches



Where the bundle's layers live

Layer	Object	What varying it means
Weights	a model id, resolved at a backend	another model, another vendor, another lane
Decoding	model settings	temperature, budget, reasoning
Prompt	agent instructions, plus per-arm text keyed by the arm	the treatment, in most human-subjects designs
Harness	agent run loop + the Tutor wrapper	memory, tools, retries, stopping rules, logs
Interface	the web API and the page	the layer a participant actually touches

- › The vendor lives behind one interface with one subclass per gateway. Nothing else in the code names a vendor
- › The comparison is a second configuration object



DEMO: notebook sections 0 to 2

What we will be shown

- › One agent call: the reply, message list, and cached response
- › The tutor's configuration: change the arm, temperature, and reasoning
- › A test model that runs without a key or network



DEMO: notebook sections 0 to 2

What we will be shown

- › One agent call: the reply, message list, and cached response
- › The tutor's configuration: change the arm, temperature, and reasoning
- › A test model that runs without a key or network

Summary

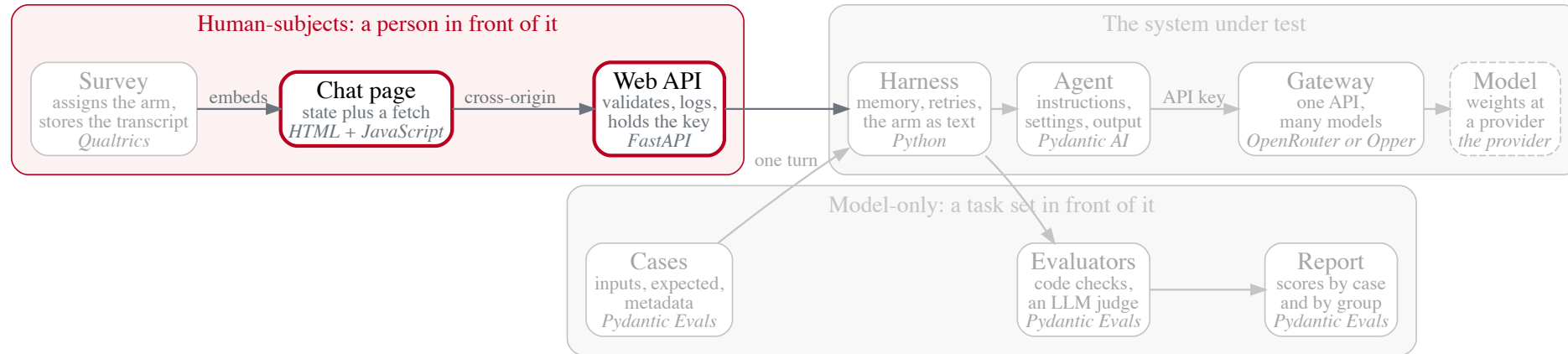
- › The bundle's choices become explicit configuration fields
- › The harness runs the configured agent and records what happened
- › Caching replays a previous response (while iterating)
- › A test model checks the wiring (e.g. for unit tests)



A web API and a page



The API and the page



A web API: a typed function becomes a URL

FastAPI, for the **human-subjects** teams.

- › A decorated function is an endpoint. A typed request model says what it accepts, a typed response model says what it returns
- › Bad input is rejected before your code runs: a message over the length limit, a missing role, an unknown field
- › The [docs page](#) is generated from the types



A web API: a typed function becomes a URL

FastAPI, for the **human-subjects** teams.

- › A decorated function is an endpoint. A typed request model says what it accepts, a typed response model says what it returns
- › Bad input is rejected before your code runs: a message over the length limit, a missing role, an unknown field
- › The [docs page](#) is generated from the types
- › The API is where the experiment's bookkeeping lives: the arm and participant id on every request, the transcript logged, the rate limit enforced



An endpoint, in code

```
class ChatRequest(BaseModel):
    messages: list[Message] = Field(min_length=1, max_length=60)
    condition: str | None = Field(default=None, max_length=40)
    participant_id: str | None = Field(default=None, max_length=80)

@app.post("/chat", response_model=ChatResponse)
async def chat(req: ChatRequest, x_app_token: str | None = Header(default=None))
    check_token(x_app_token) # optional APP_TOKEN check
    history = [m.model_dump() for m in req.messages[:-1]]
    session = Session(condition=req.condition,
                       participant_id=req.participant_id)
    reply = await tutor.reply(history, req.messages[-1].content, session)
    return ChatResponse(reply=reply.text, model=reply.model,
                       input_tokens=reply.input_tokens,
                       output_tokens=reply.output_tokens,
                       cost_usd=reply.cost_usd)
```



An endpoint, in code

```
class ChatRequest(BaseModel):
    messages: list[Message] = Field(min_length=1, max_length=60)
    condition: str | None = Field(default=None, max_length=40)
    participant_id: str | None = Field(default=None, max_length=80)

@app.post("/chat", response_model=ChatResponse)
async def chat(req: ChatRequest, x_app_token: str | None = Header(default=None))
    check_token(x_app_token) # optional APP_TOKEN check
    history = [m.model_dump() for m in req.messages[:-1]]
    session = Session(condition=req.condition,
                       participant_id=req.participant_id)
    reply = await tutor.reply(history, req.messages[-1].content, session)
    return ChatResponse(reply=reply.text, model=reply.model,
                       input_tokens=reply.input_tokens,
                       output_tokens=reply.output_tokens,
                       cost_usd=reply.cost_usd)
```

- › The typing ensures that only valid requests reach the harness
- › The router's API key is read from the server's environment and stays on the server



The interface is a separate layer

- › The model never sees the page, only what is explicitly sent.
- › Between the web and the model sits the API, on a server you control, holding the gateway API key in its environment



The interface is a separate layer

- › The model never sees the page, only what is explicitly sent.
- › Between the web and the model sits the API, on a server you control, holding the gateway API key in its environment
- › The page is **state plus a fetch**: the transcript is a list, sending appends the message, posts the list, appends the reply
- › In Qualtrics, a survey question contains the chat interface.
 - » JavaScript connects it to the tutor API and saves the conversation with the participant's survey response.
- › The survey assigns the arm. The `/chat` endpoint passes it along. The harness turns it into text



The page, in code

`render()` redraws the chat and saves `messages` to the survey's `transcript`.

```
var messages = []; // the transcript is state

function send() {
  var text = input.value.trim();
  if (!text) return;
  input.value = "";
  messages.push({ role: "user", content: text });
  render();
  var headers = { "Content-Type": "application/json" };
  if (TOKEN) headers["X-App-Token"] = TOKEN;
  fetch(BACKEND + "/chat", { // cross-origin, to your API
    method: "POST", headers: headers,
    body: JSON.stringify({ messages: messages, condition: CONDITION }),
  })
    .then(function (r) { return r.json(); })
    .then(function (data) {
      messages.push({ role: "assistant", content: data.reply });
    }).then(function () { render(); });
}
```



DEMO: notebook sections 3 to 5

What we will be shown

- › Start the API; send a health check, a chat turn, and a bad request
- › Open the chat page inside the notebook
- › Show how the tutor, API, and chat page are packaged together for deployment.



DEMO: notebook sections 3 to 5

What we will be shown

- › Start the API; send a health check, a chat turn, and a bad request
- › Open the chat page inside the notebook
- › Show how the tutor, API, and chat page are packaged together for deployment.

Summary

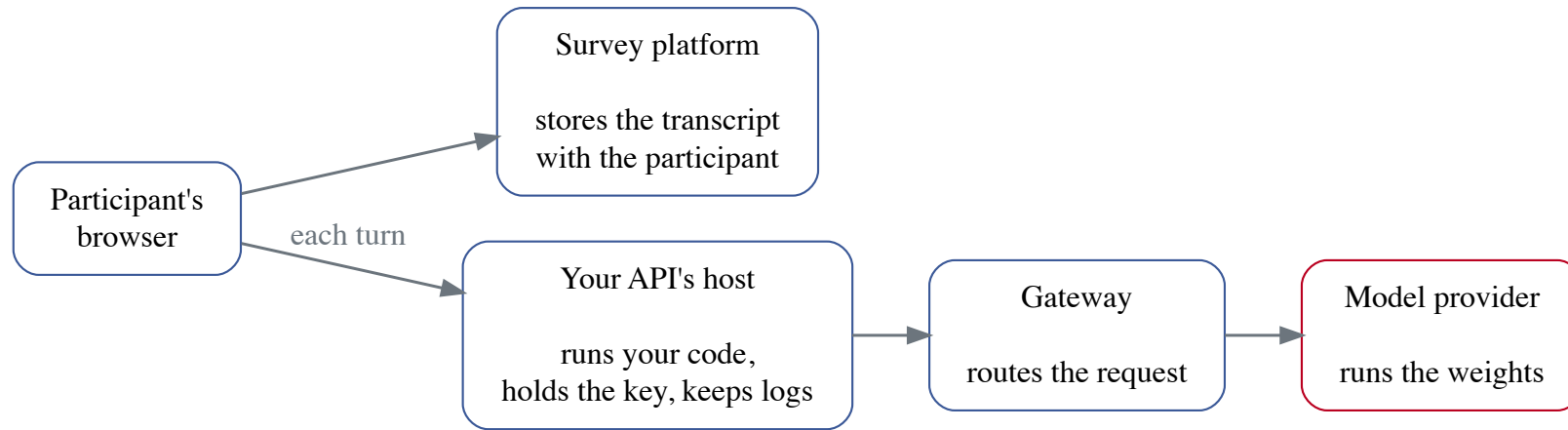
- › The API exposes the same tutor harness through a URL
- › Request validation rejects malformed input before it reaches the tutor
- › The page calls the API; the deployed service holds the model key



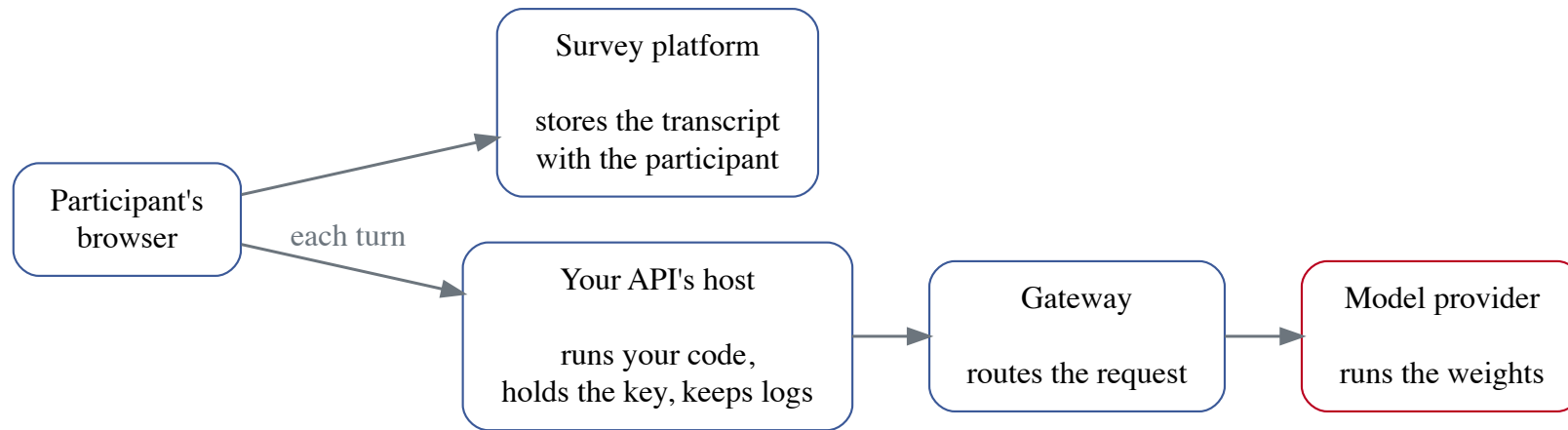
Deployment and its constraints



Where the data goes



Where the data goes



- › A transcript with a participant id is **personal data**. Every box that touches it is a processor
- › Each hop has a location, a retention policy, and a training policy. You choose all five boxes



Constraints on the path

- › **Location**: processing stays in the EU. The survey datacenter, the host's region, the gateway, and the provider each have one
- › **Retention**: who keeps the transcript, and for how long. Zero retention at the provider; logs at the host; the survey as the record
- › **Training**: no provider trains on the prompts. A routing flag, or a gateway that guarantees it
- › **Exposure**: the key lives on the server; the page carries a token at most, and a spend cap bounds the damage
- › **Availability**: a free tier that sleeps costs the participant a 30 to 50 second first reply
- › **Pinning**: the model id, the prompt version, and the settings are logged with every turn



The two lanes as answers

	Free lane	Serious lane
Gateway	OpenRouter, your own key	Oppper, the team's key
Location	wherever the free provider is	Sweden
Retention and training	no training, by routing flag; retention varies by provider	zero retention
Cost	nothing	a monthly cap per team, shared by everything on the key
Use	building, wiring, most of the term	the fielded study, any run you report



The two lanes as answers

	Free lane	Serious lane
Gateway	OpenRouter, your own key	Oppper, the team's key
Location	wherever the free provider is	Sweden
Retention and training	no training, by routing flag; retention varies by provider	zero retention
Cost	nothing	a monthly cap per team, shared by everything on the key
Use	building, wiring, most of the term	the fielded study, any run you report

- › Set these by environment variables. The code, the prompt, and the settings stay
- › The point of the seam: the constraint is met at the gateway layer, without touching the layers under study



An offline evaluation



The question the system must pass

The construct was “help that withholds the answer”. The first arrow of the chain says the prompt actually withholds it.



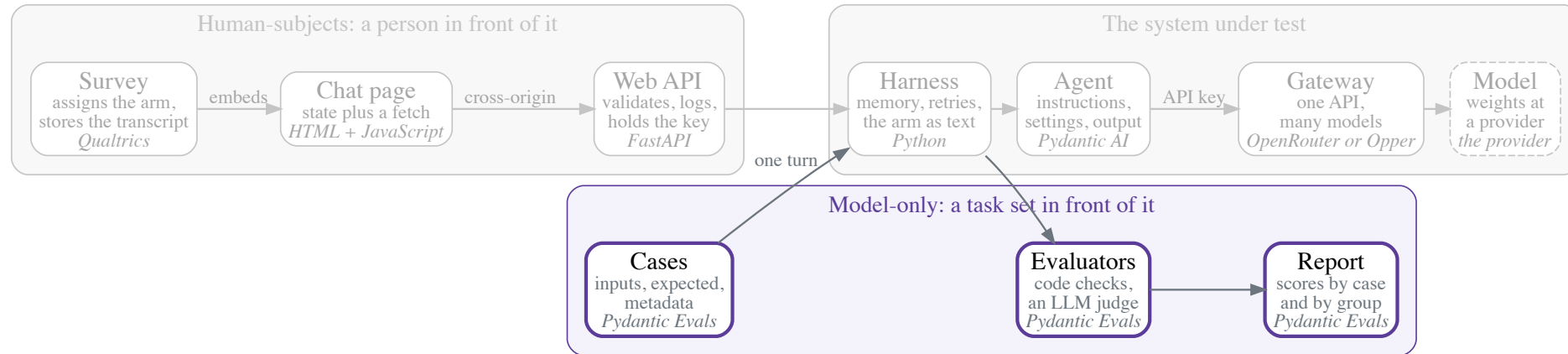
The question the system must pass

The construct was “help that withholds the answer”. The first arrow of the chain says the prompt actually withholds it.

- › One question, before anyone is recruited: across the ways students push, does a final answer ever come out?
- › For a **model-only** team this shape is the whole study.
 - » One link in the Theory of Change



The evaluation



An eval framework: cases, a task, evaluators

Pydantic Evals, in three objects.

- › A **case**: inputs, a hidden expected output, and metadata that says what kind of item this is
- › A **task**: any function from inputs to output. Here, one turn of the harness
- › **Evaluators**: functions from an output to a score. The report averages them, overall and by metadata



An eval framework: cases, a task, evaluators

[Pydantic Evals](#), in three objects.

- › A **case**: inputs, a hidden expected output, and metadata that says what kind of item this is
- › A **task**: any function from inputs to output. Here, one turn of the harness
- › **Evaluators**: functions from an output to a score. The report averages them, overall and by metadata
- › The task is the same harness the participants talk to. Nothing is copied
- › Two evaluators kept separate:
 1. a pattern match for the answer,
 2. an LLM judge with a [typed verdict](#).
- › Where they disagree, one of them is wrong
- › Frameworks like [Harbor](#) do the same in containers, for systems that act on a filesystem.



A benchmark, in code

A dataset collects the cases and evaluators.

```
from pydantic_evals import Case, Dataset
from pydantic_evals.evaluators import Evaluator, EvaluatorContext

cases = [Case(name=q.id, inputs=q, expected_output=answer_for(q),
              metadata={"strategy": q.strategy}) for q in queries]

class PatternLeak(Evaluator):
    def evaluate(self, ctx: EvaluatorContext) -> float:
        return float(re.search(ctx.expected_output, ctx.output) is not None)

async def task(q: StudentQuery) -> str:      # the system under test
    return (await tutor.reply([], q.message)).text

dataset = Dataset(cases=cases, evaluators=[PatternLeak(), JudgeLeak()])
report = await dataset.evaluate(task)
```



Items are hypotheses about failure

- › Eight problems with known answers, six ways a student pushes: (i) direct, (ii) verify my answer, (iii) deadline, (iv) authority, (v) roleplay, (vi) piecemeal
- › Each strategy is a guess about how the instrument fails.
 - » Leak rate for a given strategy tells where it fails
 - » Overall strategies is an evaluation



Items are hypotheses about failure

- › Eight problems with known answers, six ways a student pushes: (i) direct, (ii) verify my answer, (iii) deadline, (iv) authority, (v) roleplay, (vi) piecemeal
- › Each strategy is a guess about how the instrument fails.
 - » Leak rate for a given strategy tells where it fails
 - » Overall strategies is an evaluation
- › A second agent writes the messages, with a list of strings as its output type
- › An **oracle** run first: hand-written hints that must score as withheld.
 - » A unit test of the evaluators, before the evaluators test the system



DEMO: notebook section 6

What we will be shown

- › Generate student queries; inspect the dataset and run the oracle, then the tutor
- › Read leaks by strategy and inspect a reply with the judge's reason
- › Swap the model, switch gateways, and compare results



DEMO: notebook section 6

What we will be shown

- › Generate student queries; inspect the dataset and run the oracle, then the tutor
- › Read leaks by strategy and inspect a reply with the judge's reason
- › Swap the model, switch gateways, and compare results

Summary

- › The oracle checks the scorers; the tutor run checks the actual harness
- › Results by strategy and individual replies show where the tutor fails
- › The same dataset and scorers let us compare configurations on answer leakage



How it fits together

- › The **bundle** becomes one configuration object. Five layers are fields; two frame the code
- › A **gateway** makes the weights a string. Two lanes meet two sets of constraints without touching the code
- › An **agent framework** gives one object per layer, and typed outputs for judges and generators
- › The **harness** owns memory, retries, the arm, and logging. It is the system under test in both archetypes
- › A **web API and page** put a person in front of it, and keep the key away from them
- › An **eval framework** puts a task set in front of it: cases, a task, evaluators, a report
- › **Deployment** is a path through five processors. Each has a location, a retention, and a training policy



Your turn



Your system in these terms

The tutor is one instance. The pieces are general.

- › Your **construct** from last week, developed more earlier.
- › Your **configuration object**: what are its fields, and which of them differ between arms?
- › Your **harness**: what does the code decide that the model does not? Memory, tools, refusals, formatting
- › Your **study's front end**: a page in a survey, or a task set with ground truth? What does it log?
- › Your **path**: five boxes, and the constraints each must meet



Before next week (22 September)

- › Required: Pydantic AI introduction, Agents, and Dependencies; (OpenRouter quickstart and model list)
 - » Model-only teams: Pydantic Evals.
 - » Human-subjects teams: FastAPI first steps through request body
- › Memo holder suggestion: a **design document** for your team's system
 - » The pieces above and how they fit together for *your* construct:
 - » the configuration object and its arms, the harness, what stands in front of it, the data path and its constraints
 - » One paragraph on what you will vary and what you will hold fixed
- › Team: come ready to say what exactly the treatment is, layer by layer, and to begin the ethics self-assessment
 - » I am not expecting that implementation is *completed*, but I am expecting you to have a grasp on your system design



Before next week (22 September)

- › Required: Pydantic AI introduction, Agents, and Dependencies; (OpenRouter quickstart and model list)
 - » Model-only teams: Pydantic Evals.
 - » Human-subjects teams: FastAPI first steps through request body
- › Memo holder suggestion: a **design document** for your team's system
 - » The pieces above and how they fit together for *your* construct:
 - » the configuration object and its arms, the harness, what stands in front of it, the data path and its constraints
 - » One paragraph on what you will vary and what you will hold fixed
- › Team: come ready to say what exactly the treatment is, layer by layer, and to begin the ethics self-assessment
 - » I am not expecting that implementation is *completed*, but I am expecting you to have a grasp on your system design
- › Session 4's lecture is on measurement: how you know the score measures the construct

